

REPORT

과제물 주제		데이터베이스를 구성하는 테이블 생성과 질의문 작성
1차	2차	

성 명	정주호		학습과정명	데이터베이스실습	
학 번	225230001		교 · 강사명	한인	
전공명 / 학급명	컴퓨터공학 전공	AA	취득점수 / 교 · 강사확인		

1. 개요 및 문제 설명

1.1) 과제 목적

본 과제 주제는 데이터베이스 실습에서 학습한 DDL, DML 명령을 이용하여 학생 정보 테이블과 전공 정보 테이블을 생성하고, 기본키와 외래키를 포함한 제약조건을 설정한 뒤 실제 데이터를 저장, 조회, 수정하는 과정을 확인하는 것입니다.

tblDuicaDept 테이블은 전공 정보를 관리하고, tblDuicaSt 테이블은 학생 정보를 관리합니다. 두 테이블은 tblDuicaSt.MAJOR 열이 tblDuicaDept.DEPTNO 열을 참조하도록 외래키로 연결됩니다. 이를 통해 관계형 데이터베이스에서 기본적으로 지켜야 할 데이터 구조 정의, 제약조건 설정, 데이터 입력, 조회, 수정의 흐름을 직접 확인할 수 있습니다. 특히 두 테이블의 관계를 기준으로 전공 정보가 먼저 정의되고, 학생 정보가 그 전공 정보를 참조하는 구조를 구성함으로써 테이블 간 관계가 데이터 저장 과정에서 어떤 기준으로 작동하는지 이해할 수 있습니다.

또한 기본키, 외래키, NOT NULL 제약조건을 설정하여 각 데이터가 식별 가능하고, 필수 값이 누락되지 않으며, 서로 관련된 테이블 간 데이터가 일관성을 유지하도록 구성합니다. 이후 INSERT 문으로 데이터를 저장하고, SELECT 문으로 저장된 결과를 조회하며, UPDATE 문으로 특정 값을 수정하는 과정을 수행함으로써 DDL, DML 명령이 실제 데이터베이스에서 어떻게 연결되어 사용되는지 확인할 수 있습니다. 따라서 본 과제는 '데이터베이스를 구성하는 테이블 생성과 질의문 작성'이라는 주제에 따라, 테이블을 작성하고 제약조건을 설정한 뒤 질의문을 작성하여 실행 결과를 검증하는 전체 과정을 정리하는 데 목적이 있습니다.

1.2) 핵심 요구사항

- I. tblDuicaDept 테이블과 tblDuicaSt 테이블을 생성한다.
- II. tblDuicaDept의 DEPTNO는 기본키, DNAME은 NOT NULL 제약조건으로 설정한다.
- III. tblDuicaSt의 SNO는 기본키, SNAME과 HIREDATE는 NOT NULL 제약조건으로 설정한다.
- IV. tblDuicaSt의 MAJOR는 tblDuicaDept의 기본키인 DEPTNO를 참조하는 외래키로 설정한다.
- V. 전공번호 10, 전공이름 '컴퓨터공학' 데이터와 본인의 학생 정보를 INSERT 한다.
- VI. 저장된 학생 이름을 홍길동으로 UPDATE 한 뒤 결과를 조회한다.
- VII. USER_CONSTRAINTS를 이용하여 두 테이블에 설정된 제약조건을 확인한다.

1.3) 핵심 개념

이번 과제에서 사용한 SQL은 크게 세 가지 흐름으로 나눌 수 있습니다. 첫째, CREATE TABLE과 ALTER TABLE을 사용하여 테이블 구조를 정의하는 DDL입니다. 둘째, INSERT와 UPDATE를 사용하여 데이터를 저장하고 수정하는 DML입니다. 셋째, 테이블에 저장된 데이터와 제약조건 정보만을 따로 조회할 수 있는 SELECT 문의 활용입니다.

특히 기본키, NOT NULL, 외래키 제약조건을 함께 사용하여 테이블이 단순히 값을 저장하는 공간이 아니라 데이터 정확성과 관계를 유지하는 구조라는 점을 확인하였습니다.

SQL 실행 흐름

CREATE TABLE → INSERT → SELECT → UPDATE → USER_CONSTRAINTS



1.4) 성공 기준

- 1) 두 테이블이 오류 없이 생성되어야 한다.
- 2) 전공 테이블에 DEPTNO 10, DNAME '컴퓨터공학전공'이 성공적으로 저장되어야 한다.
- 3) 학생 테이블에 SNO 225230001, SNAME '정주호', HIREDATE '2025-09-01', MAJOR 10 데이터가 성공적으로 저장되어야 한다.
- 4) UPDATE 수행 후 SNAME이 '홍길동'으로 변경되어야 한다.
- 5) USER_CONSTRAINTS 조회 결과에서 기본키, NOT NULL, 외래키 제약조건이 확인되어야 한다.

검증 항목	확인 방법	성공 판단
테이블 생성	CREATE TABLE 실행 후 오류 메시지 확인	오류 없이 생성
데이터 저장	SELECT로 INSERT 결과 조회	입력한 값과 동일
데이터 수정	UPDATE 후 SELECT로 변경 결과 조회	이름이 홍길동으로 변경
제약조건	USER_CONSTRAINTS 조회	P, C, R 유형의 제약조건 확인

1.5) 실습 환경

본 과제를 진행하기 위한 실습환경으로 DBMS는 Oracle Database 11g Enterprise Edition을 사용하였으며, 데이터베이스 관리 도구로는 DBeaver 26.0 버전을 활용하였습니다. 또한 데이터베이스 접속 및 실습을 위한 사용자 계정으로는 scott 계정을 사용하였습니다.

2. 테이블 설계 및 구조 파악

2.1) 테이블 구조 설계

[tblDuicaDept]

열 이름	자료형	길이	설명	제약조건
DEPTNO	NUMBER	2, 0	전공 번호	PRIMARY KEY
DNAME	VARCHAR	16	전공 이름	NOT NULL

tblDuicaDept 테이블은 전공 번호와 전공 이름을 저장합니다. DEPTNO는 전공을 구분하는 기준 값이므로 기본키로 지정하고, DNAME은 전공 이름이 비어 있으면 안 되므로 NOT NULL 제약조건을 설정합니다.

[tblDuicaSt]

열 이름	자료형	길이	설명	제약조건
SNO	NUMBER	8, 0 → 9, 0	학번	PRIMARY KEY
SNAME	VARCHAR	8	이름	NOT NULL
HIREDATE	DATE		입학일	NOT NULL
MAJOR	NUMBER	2, 0	전공학과 참조 번호	FOREIGN KEY

tblDuicaSt 테이블은 학생의 학번, 이름, 입학일, 전공 번호를 저장합니다. SNO는 학생을 구분하는 고유 값이므로 기본키로 설정하고, SNAME과 HIREDATE는 필수 입력 정보이므로 NOT NULL로 설정합니다. MAJOR는 전공 번호를 나타내며, tblDuicaDept 테이블의 DEPTNO를 참조하도록 외래키로 설정합니다.

과제에서 제시된 SNO의 길이가 8로 제시되어 있지만 본인의 학번은 225230001로 9자리입니다. 따라서 실제 데이터 저장 전에 ALTER TABLE 문으로 SNO 열의 길이를 NUMBER(9,0)으로 확장하여 학번이 정상적으로 저장되도록 해야 합니다.

테이블 참조 관계

tblDuicaSt.MAJOR 값은 tblDuicaDept.DEPTNO에 존재해야 함



2.2) ER 다이어그램 및 릴레이션 스키마

ER 다이어그램

전공 1개는 여러 학생과 연결되고, 학생 1명은 하나의 전공을 가짐



릴레이션 스키마

기본키 · 외래키 · 필수 입력 속성을 기준으로 테이블 구조 정리



참조식: tblDuicaSt.MAJOR → tblDuicaDept.DEPTNO

위 ER 다이어그램은 전공 릴레이션과 학생 릴레이션 사이의 1:N 관계를 시각화한 것입니다. 전공 번호 DEPTNO는 전공을 식별하는 기본키이며, 학생 테이블의 MAJOR은 이 값을 참조하는 외래키입니다.

릴레이션 스키마는 같은 구조를 관계형 데이터베이스 관점에서 정리한 것입니다. 각 속성의 역할을 PK, FK, NN으로 구분하여 SQL 제약조건과 논리 설계가 어떻게 연결되는지 확인할 수 있도록 하였습니다.

2.3) 정규형 검토

두 릴레이션은 속성이 원자값으로 구성되어 반복 그룹이 없으므로 1정규형을 만족합니다. 또한 두 테이블의 기본키가 각각 단일 속성이기 때문에 비키 속성이 기본키의 일부에만 의존하는 부분 함수 종속이 발생하지 않아 2정규형도 만족합니다.

3정규형 관점에서는 비키 속성이 다른 비키 속성을 결정하는 이행 종속이 없어야 합니다. 본 설계에서는 학생 테이블에 전공명 DNAME을 직접 저장하지 않고 MAJOR만 저장한 뒤 tblDuicaDept를 참조하므로, 전공명 변경이 필요할 때 학생 테이블의 여러 행을 반복 수정하지 않아도 됩니다.

따라서 현재 함수 종속 기준으로 전공 릴레이션과 학생 릴레이션은 최소 3정규형을 만족하며, 각 릴레이션에서 의미 있는 결정자가 후보키이므로 BCNF 수준으로도 해석할 수 있습니다. 단 실제 업무에서는 한 학생이 여러 전공을 동시에 가질 수 있다면 학생-전공 교차 릴레이션을 추가해 다대다 관계를 분리하는 설계가 필요합니다.

3. SQL 구현

3.1) 두 개 테이블 생성 SQL

전공 테이블을 먼저 생성한 뒤 학생 테이블을 생성하였습니다. 학생 테이블의 외래키가 전공 테이블의 기본키를 참조하므로, 참조되는 테이블인 tblDuicaDept가 먼저 존재해야 합니다.

```
CREATE TABLE "tblDuicaDept"(  
    DEPTNO NUMBER(2,0) CONSTRAINT tblDuicaDept_DEPTNO_PK PRIMARY KEY,  
    DNAME VARCHAR(16) CONSTRAINT tblDuicaDept_DNAME_NN NOT NULL  
);  
  
CREATE TABLE "tblDuicaSt"(  
    SNO NUMBER(8,0) CONSTRAINT tblDuicaSt_SNO_PK PRIMARY KEY,  
    SNAME VARCHAR(8) CONSTRAINT tblDuicaSt_SNAME_NN NOT NULL,  
    HIREDATE DATE CONSTRAINT tblDuicaSt_HIREDATE_NN NOT NULL,  
    MAJOR NUMBER(2, 0) CONSTRAINT tblDuicaSt_MAJOR_FK REFERENCES "tblDuicaDept"(DEPTNO)  
);
```

각 테이블은 행을 고유하게 식별하기 위한 기본키 제약조건인 PRIMARY KEY, 필수 입력값을 보장하기 위한 NOT NULL 제약 조건, 학생의 전공 번호가 전공 테이블에 존재하는 번호만 가질 수 있도록 하는 외래키 제약조건을 설정받았습니다.

Oracle 데이터베이스의 특성상 식별자에 큰따옴표를 사용하지 않으면 자동으로 대문자로 변환되어 처리됩니다. 따라서 요구된 테이블 명의 대소문자를 엄밀하게 유지하고 오작동을 방지하기 위해, 모든 실습 과정에서 해당 테이블 이름에 항상 큰따옴표(" ")를 지정하여 SQL 스크립트를 작성하였습니다.

```
SELECT *  
FROM "tblDuicaDept";  
  
SELECT *  
FROM "tblDuicaSt";
```

테이블이 정상적으로 생성되었는지 신속하게 확인하기 위한 목적이므로, 특정 필드를 지정하여 프로젝트하는 대신 전체 열을 출력하도록 애스터리스크(*)를 사용하여 데이터를 조회하였습니다.

1. CREATE TABLE 실행 결과

tblDuicaSt Updated Row: 0 Deleted Row: 0 Start Time: Sun Jun 07 00:03:41 KST 2020 Finish Time: Sun Jun 07 00:03:41 KST 2020 Query: CREATE TABLE "tblDuicaSt"(SNO NUMBER(8,0) CONSTRAINT "tblDuicaSt_SNO_PK" PRIMARY KEY, SNAME VARCHAR(8) CONSTRAINT "tblDuicaSt_SNAME_NN" NOT NULL, HIREDATE DATE CONSTRAINT "tblDuicaSt_HIREDATE_NN" NOT NULL, MAJOR NUMBER(2,0) CONSTRAINT "tblDuicaSt_MAJOR_FK" REFERENCES "tblDuicaDept"(	tblDuicaDept Updated Row: 0 Deleted Row: 0 Start Time: Sun Jun 07 00:03:39 KST 2020 Finish Time: Sun Jun 07 00:03:39 KST 2020 Query: CREATE TABLE "tblDuicaDept"(DEPTNO NUMBER(2,0) CONSTRAINT "tblDuicaDept_DEPTNO_PK" PRIMARY KEY, DNAME VARCHAR(16) CONSTRAINT "tblDuicaDept_DNAME_NN" NOT NULL
---	---

2. 테이블 구조 및 참조 관계 확인



3. SELECT 조회 결과

Result 1: SELECT * FROM "tblDuicaDept" DEPTNO DNAME 10 ACCOUNTING 20 MARKETING 30 SALES 40 OPERATIONS 50 TRAINING	Result 2: SELECT * FROM "tblDuicaSt" SNO SNAME HIREDATE MAJOR 1001 SCOTT 1987-06-07 10 1002 ADAMS 1989-04-12 30 1003 JONES 1981-02-01 20 1004 SMITH 1980-09-14 40 1005 ALLEN 1983-09-14 30 1006 WATSON 1985-02-09 40 1007 SCOTT 1987-06-07 10 1008 JONES 1981-02-01 20 1009 SMITH 1980-09-14 40 1010 ALLEN 1983-09-14 30 1011 WATSON 1985-02-09 40
--	---

3.2) 전공 정보와 학생 정보 저장 SQL

과제 요구조건에 따라 전공 테이블에 전공번호 10과 '컴퓨터공학전공' 값을 먼저 저장하였습니다. 이어서 학생 테이블에 본인의 학번, 이름, 입학일, 전공번호를 저장해야 합니다.

다만, 앞서 서술했듯이 본인의 학번은 9자리 정수형인 반면, 기존 학생 테이블의 학번(SNO) 열은 정수형 8자리로 설계되어 있습니다. 따라서 해당 열의 길이 제한을 먼저 수정해야만 정상적으로 학생 정보를 저장할 수 있습니다. 만약 테이블 구조를 수정하지 않고 INSERT를 수행한다면, "ORA-01438: 이 열에 대해 지정된 전체 자릿수보다 큰 값이 허용되지 않습니다"라는 에러가 발생합니다.

또한, 본인의 전공은 전공 테이블에 이미 등록한 '컴퓨터공학전공'이므로, 참조할 전공번호 데이터에 10을 지정하여 외래키(Foreign Key) 제약조건을 만족하며 데이터를 안전하게 저장할 수 있습니다. 관계형 데이터베이스의 특성상, 만약 다른 전공 데이터를 입력하고자 했다면 해당 전공 데이터를 전공 테이블(부모 테이블)에 먼저 참조 무결성에 맞게 저장해야 합니다. 그렇지 않고 자식 테이블인 학생 테이블에 바로 INSERT를 시도하면, "ORA-02291: 무결성 제약조건이 위배되었습니다 - 부모 키가 없습니다"라는 제약조건 위배 에러를 반환하게 됩니다.

```
INSERT INTO "tblDuicaDept" (DEPTNO, DNAME)
VALUES (10, '컴퓨터공학전공');
```

```
ALTER TABLE "tblDuicaSt"
MODIFY SNO NUMBER(9,0);
```

```
INSERT INTO "tblDuicaSt" (SNO, SNAME, HIREDATE, MAJOR)
VALUES (225230001, '정주호', TO_DATE('2025-09-01', 'YYYY-MM-DD'), 10);
```

1. 전공 데이터 입력	2. 학생 학번 자릿수 수정
Statistics 1 Name Value Updated Rows 1 Execute time 0.026s Start time Sun Jun 07 21:17:16 KST 2026 Finish time Sun Jun 07 21:17:16 KST 2026 Query INSERT INTO "tblDuicaDept" (DEPTNO, DNAME) VALUES (10, '컴퓨터공학전공')	Statistics 1 Name Value Updated Rows 0 Execute time 0.009s Start time Sun Jun 07 21:19:54 KST 2026 Finish time Sun Jun 07 21:19:54 KST 2026 Query ALTER TABLE "tblDuicaSt" MODIFY SNO NUMBER(9,0)

3. 학생 데이터 입력

Statistics 1 Name Value Updated Rows 1 Execute time 0.001s Start time Sun Jun 07 21:21:41 KST 2026 Finish time Sun Jun 07 21:21:41 KST 2026 Query INSERT INTO "tblDuicaSt" (SNO, SNAME, HIREDATE, MAJOR) VALUES (225230001, '정주호', TO_DATE('2025-09-01', 'YYYY-MM-DD'), 10)	
---	--

3.3) 정보 저장 결과 확인 SQL

INSERT가 정상적으로 수행되었는지 확인하기 위해 두 테이블을 각각 SELECT 문으로 조회합니다. 학생 테이블의 학번은 숫자형으로 저장되어있지만 화면에서는 앞뒤 형식이 명확하게 보이도록 TO_CHAR 함수를 사용하여 문자형으로 표시하여 확인했습니다. JOIN을 시켜서 조회하면 더 읽기 쉬운 결과를 얻을 수 있습니다.

```
SELECT DEPTNO, DNAME
FROM "tblDuicaDept";
```

```
SELECT TO_CHAR(SNO) AS SNO, SNAME, HIREDATE, MAJOR
FROM "tblDuicaSt";
```

```
SELECT DEPTNO, DNAME, TO_CHAR(SNO) AS SNO, SNAME, HIREDATE, MAJOR
FROM "tblDuicaDept" D JOIN "tblDuicaSt" S ON (D.DEPTNO = S.MAJOR);
```

DEPTNO	DNAME	SNO	SNAME	HIREDATE	MAJOR
10	컴퓨터공학전공	225230001	정주호	2025-09-01 00:00:00.000	10

3.4) 학생 이름 변경 SQL

학생 테이블에 저장된 본인의 이름을 홍길동으로 변경하기 위해 UPDATE 문을 사용하였습니다. WHERE 절을 생략하면 테이블의 모든 행이 수정될 수 있습니다. 본 과제에서 테이블에 삽입된 정보는 하나밖에 없지만, 수정 범위에 맞게 조건을 제한시켜 필요한 학생 정보만 변경하는게 적절합니다. WHERE 조건에는 기본키인 SNO를 사용하여 본인의 정보 중 학번이 일치하는 경우로 설정하여 변경 대상이 되는 행을 정확하게 지정했습니다.

```
UPDATE "tblDuicaSt"
SET SNAME = '홍길동'
WHERE SNO = 225230001;
```

UPDATE 실행 결과

Name	Value
Updated Rows	1
Execute time	0.003s
Start time	Sun Jun 07 21:34:30 KST 2026
Finish time	Sun Jun 07 21:34:30 KST 2026
Query	UPDATE "tblDuicaSt" SET SNAME = '홍길동' WHERE SNO = 225230001

3.5) 이름 변경 결과 확인 SQL

UPDATE 후 동일한 학번을 SELECT 문으로 다시 조회하여 SNAME 값이 '정주호'에서 '홍길동'으로 변경되었는지 확인할 수 있습니다.

```
SELECT TO_CHAR(SNO) AS SNO, SNAME, HIREDATE, MAJOR
FROM "tblDuicaSt"
WHERE SNO = 225230001;
```

```
SELECT TO_CHAR(SNO) AS SNO, SNAME, HIREDATE, MAJOR
FROM "tblDuicaSt"
WHERE SNAME = '홍길동';
```

특정 학생 정보 데이터를 조회할 때 다양한 조건으로, 원하는 대로 WHERE 조건을 줄 수 있지만 기본키를 사용하는 것이 적절합니다. 기본키는 테이블에서 각 행을 유일하게 식별하기 위한 속성으로 중복될 수 없고 NULL 값을 가질 수 없기 때문에, 기본키를 조건으로 지정하는 것이 더 명확하게 조회 대상을 식별하고 중복된 결과를 발생시키지 않을 수 있습니다. 또한 일반적으로 기본키에 인덱스가 생성되므로 전체 테이블을 처음부터 끝까지 검색하는 방식보다 효율적으로 데이터를 찾을 수 있어 검색 성능 측면에서도 유리합니다.

UPDATE 후 SELECT 조회 결과

SELECT TO_CHAR(SNO) AS SNO, SNAME, HIREDATE, MAJOR				
	A-Z SNO	A-Z SNAME	HIREDATE	123 MAJOR
1	225230001	홍길동	2025-09-01 00:00:00.000	10

3.6) 제약조건 확인 SQL

USER_CONSTRAINTS 데이터 사전 뷰를 조회하면 현재 사용자가 소유한 테이블의 모든 제약조건 정보를 상세히 확인할 수 있습니다. 이 뷰의 전체적인 열 구성을 파악한 뒤 요구사항에 맞게 조건절을 작성하면, 생성한 특정 두 개의 테이블에 대한 제약조건만 정확하게 필터링하여 조회하는 것이 가능합니다. 구체적으로는 USER_CONSTRAINTS 뷰 내부에서 TABLE_NAME이라는 필드가 해당 제약조건이 속한 테이블의 이름을 저장하고 있기 때문에, 원하는 테이블의 정보만 선별하기 위해서는 이 필드 이름을 대상으로 조건절의 프레디키트를 수행해야 합니다.

이때 조건절을 작성하는 방식은 테이블을 처음 생성할 때 정의했던 방식에 따라 달라집니다. 본 실습 과정에서는 테이블 이름을 생성할 때 큰따옴표를 지정하여 대소문자를 명확히 구분하도록 설정했기 때문에, 다중행 함수인 IN 연산자 내에 테이블 이름을 문자열 형태로 있는 그대로 입력하여 조회하면 됩니다. 그러나 만약 테이블 생성 시 큰따옴표를 지정하지 않았다면, 일반적인 조회나 갱신 연산을 수행할 때와 달리 데이터 사전 뷰의 관점에서는 테이블 이름 자체가 하나의 엄격한 데이터로 취급되어 대소문자를 철저히 구분하므로 반드시 UPPER 함수를 사용하여 대문자로 변환한 뒤 조회 조건에 매칭시켜야 합니다.

```
SELECT *
FROM USER_CONSTRAINTS;
```

```
SELECT *
FROM USER_CONSTRAINTS
WHERE TABLE_NAME IN ('tblDuicaSt', 'tblDuicaDept');
```

1. USER_CONSTRAINTS 전체 조회

Results 1 X

SELECT * FROM USER_CONSTRAINTS

	AZ OWNER	AZ CONSTRAINT_NAME	AZ CONSTRAINT_TYPE	AZ TABLE_NAME	SEARCH_CONDITION	AZ R_OWNER
1	SCOTT	FK_DEPTNO	R	EMP	[NULL]	SCOTT
2	SCOTT	TBLDUICAST_MAJOR_FK	R	tblDuicaSt	[NULL]	SCOTT
3	SCOTT	PK_DEPT	P	DEPT	[NULL]	[NULL]
4	SCOTT	PK_EMP	P	EMP	[NULL]	[NULL]
5	SCOTT	TBLDUICADEPT_DEPTNO_PK	P	tblDuicaDept	[NULL]	[NULL]
6	SCOTT	BIN\$MBqHwq/MSA2kBRUBAYBavQ== \$0	C	BIN\$069EVXQK54C409dZ9vd5qw== \$0	"SNAME" IS NOT NULL	[NULL]
7	SCOTT	BIN\$1HgZTP+XR5yVdHpgwRkW/g== \$0	C	BIN\$069EVXQK54C409dZ9vd5qw== \$0	"HIREDATE" IS NOT NULL	[NULL]
8	SCOTT	BIN\$Yq9RbDhRKQJuc3Ve48HQ== \$0	C	BIN\$clH7Y2R6+5guqKno6tA== \$0	"DNAME" IS NOT NULL	[NULL]
9	SCOTT	BIN\$e5Aa0IUUQOuOQo1Q/4HbeA== \$0	C	BIN\$g2L3W7Z1R4OdLMzR0zGMA== \$0	"LOC" IS NOT NULL	[NULL]
10	SCOTT	BIN\$F3qjstHDQnOgZChGjdxRw== \$0	P	BIN\$069EVXQK54C409dZ9vd5qw== \$0	[NULL]	[NULL]
11	SCOTT	BIN\$fr36TmSRWQg3NP2bxmipQ== \$0	C	BIN\$7LqwmfQ9y0Qk/M8+J+EQ== \$0	"DNAME" IS NOT NULL	[NULL]

2. TABLE_NAME 조건 조회

Results 1 X

SELECT * FROM USER_CONSTRAINTS WHERE TABLE_NAME

	AZ OWNER	AZ CONSTRAINT_NAME	AZ CONSTRAINT_TYPE	AZ TABLE_NAME	SEARCH_CONDITION	AZ R_OWNER	AZ
1	SCOTT	TBLDUICADEPT_DNAME_NN	C	tblDuicaDept	"DNAME" IS NOT NULL	[NULL]	[N]
2	SCOTT	TBLDUICADEPT_DEPTNO_PK	P	tblDuicaDept	[NULL]	[NULL]	[N]
3	SCOTT	TBLDUICAST_SNAME_NN	C	tblDuicaSt	"SNAME" IS NOT NULL	[NULL]	[N]
4	SCOTT	TBLDUICAST_HIREDATE_NN	C	tblDuicaSt	"HIREDATE" IS NOT NULL	[NULL]	[N]
5	SCOTT	TBLDUICAST_SNO_PK	P	tblDuicaSt	[NULL]	[NULL]	[N]
6	SCOTT	TBLDUICAST_MAJOR_FK	R	tblDuicaSt	[NULL]	SCOTT	TBL

3. 테이블별 제약조건 목록

Column	이름	Column	소유자	Type	Condition	Status
Columns	TBLDUICADEPT_DEPTNO_FK	tblDuicaDept	SCOTT	NUMBER(4)	PRIMARY KEY	ENABLED
Columns	TBLDUICAST_SNAME_NN	tblDuicaSt	SCOTT	VARCHAR2(10)	CHECK	"SNAME" IS NOT NULL
Columns	TBLDUICAST_HIREDATE_NN	tblDuicaSt	SCOTT	DATE	CHECK	"HIREDATE" IS NOT NULL
Columns	TBLDUICAST_SNO_PK	tblDuicaSt	SCOTT	NUMBER(4)	PRIMARY KEY	ENABLED

4. 외래키 및 참조 관계 확인

Column	이름	Column	소유자	Type	Ref Table	Type	Ref Table	On Delete	Status
Columns	TBLDUICAST_MAJOR_FK	tblDuicaSt	SCOTT	NUMBER(4)	TBLDUICADEPT	FOREIGN KEY	tblDuicaDept	on delete	ENABLED

4. 결론

4.1) 요구 조건 충족 여부

요구조건-실행결과 매핑

작성한 SQL 문장과 DBeaver 결과 화면을 기준으로 충족 여부 확인

요구조건	SQL 문장	확인 기준	판정
테이블 생성	CREATE TABLE	tblDuicaDept, tblDuicaSt 구조 생성	충족
제약조건 설정	PK / FK / NOT NULL	PK, FK, 필수 입력 조건 확인	충족
데이터 입력	INSERT	전공 정보와 학생 정보 저장	충족
데이터 조회	SELECT	자장던 결과 관계 결과 조회	충족
데이터 수정	UPDATE	조건에 맞는 학생 정보 변경	충족

4.2) 학습 성과

이번 과제를 통해 테이블을 생성하는 DDL 명령, 데이터를 입력·수정하는 데이터 조작(갱신 연산) 명령, 그리고 SELECT 문을 이용한 조회가 서로 다른 목적을 가진 작업이라는 점을 명확히 이해할 수 있었습니다. CREATE TABLE 단계에서는 단순히 테이블 이름만 정하는 것이 아니라, 열 이름, 자료형, 데이터 길이, 기본키, 외래키, NOT NULL 제약조건을 함께 설계해야 한다는 점을 확인하였습니다. 또한 정규형, 엔터티 간 관계, 릴레이션 스키마 등 데이터베이스 이론 시간에 학습한 내용을 실제 SQL 문장으로 작성하고 실행해 보면서, 이론적인 개념이 실제 테이블 설계 과정에서 어떻게 적용되는지 다시 정리할 수 있었습니다.

특히 기본키와 외래키, NOT NULL 제약조건은 데이터의 정확성과 일관성을 유지하기 위한 중요한 기준이라는 점을 알 수 있었습니다. 기본키는 각 행을 유일하게 식별하고, 외래키는 서로 다른 테이블 사이의 참조 관계를 유지하며, NOT NULL 제약조건은 반드시 입력되어야 하는 값이 누락되지 않도록 관리합니다. DBMS가 이러한 제약조건을 자동으로 검사하고 관리하지만, 어떤 열에 어떤 자료형과 길이를 지정할지, 어떤 열을 기본키와 외래키로 설정할지는 사용자가 설계 단계에서 신중하게 결정해야 한다는 점도 확인할 수 있었습니다.

또한 Oracle이 제약조건과 무결성을 내부적으로 어떻게 관리하는지도 USER_CONSTRAINTS 데이터 사전 뷰를 통해 확인할 수 있었습니다. 제약조건 이름, 제약조건 유형, 테이블명, 검색 조건, 참조 대상 등을 조회하면서 단순히 테이블의 행과 열만 보는 것이 아니라, 스키마, 사용자, 릴레이션 간 관계, 데이터 사전 뷰까지 확장하여 데이터베이스 구조를 이해할 수 있었습니다. 이를 통해 특정 정보가 궁금할 때 DBeaver의 객체 탭이나 Oracle의 데이터 사전 뷰를 활용하여 구조와 제약조건을 추적할 수 있다는 점을 알게 되었습니다.

결과적으로 이번 과제는 전공 정보 테이블과 학생 정보 테이블을 생성하고, 제약조건을 설정한 뒤 데이터를 입력, 조회, 수정하는 과정을 종합적으로 연습한 실습이었습니다. 보고서를 작성하면서 SQL 실행 결과를 단순히 나열하는 것이 아니라, 각 명령이 어떤 목적을 가지며 테이블 구조와 어떤 관계를 맺는지 정리할 수 있었습니다. 이후에는 CHECK 제약조건을 추가하거나, 더 많은 데이터를 입력한 뒤 학과별로 제공할 수 있는 VIEW를 만들어 보는 방식으로 확장해 볼 수 있을 것입니다.